



DIGITAL IMAGE PROCESSING TECHNIQUES FOR OBJECT DETECTIONFROM COMPLEX BACKGROUND IMAGE



SUBMITTED TO:

DR. BAMBAM KUMAR
ASST. PROFESSOR
NIT PATNA

SUBMITTED BY:

PULKIT GAUTAM	2004006
SHIVANSHU TRIPATHI	2004034
ATUL KUMAR	2004038

ABSTRACT

This project aims to detect and allocate objects using methods such as color processing and blob analysis. The program should automatically detect and count the total number of objects in an image of a mango tree. By using this method, we will be able to efficiently detect the mangos, eliminate the background image, and count the total number of objects. During the object detection process, the main problem faced in the image processing field is the targeted object being obscured due to the presence of other objects. This is due to objects not clearly expressing in the image, and the program assumes and eliminates them. Objects that overlap with each other also make the process challenging, where hidden objects are detected and counted, and the total number is inaccurate. In color processing, lighting intensity may affect the original color of the object and make the background object have almost the same color as the target object. To eliminate all these problems, proper image segmentation processes and techniques must be taken into account.

INTRODUCTION

Object detection is a computer vision technique used to locate instances of objects in images or videos. When humans look at images or video, we can recognize and locate objects of interest within moments. Object detection replicates this intelligence using a computer. Various approaches can be used to detect objects, including deep learning, machine learning, and image segmentation. Object detection is a powerful tool that can be applied to a range of applications, from facial recognition to autonomous vehicles. It can also be utilized in applications such as security surveillance, medical imaging, autonomous navigation, and more.

There are several methods that can be applied to our task, and one such example is the Color Hough Transform (CHT), which is a shape detection method that detects most of the round objects. ^[1]The academic paper we referenced uses this method to perform the same task. When comparing CHT and Blob Analysis, the Color Hough Transform is a variant of the standard Hough Transform, which is used to detect lines in an image. ^[2]The Color Hough Transform is used to detect color regions or patterns in an image, including circles, ellipses, and other geometric shapes based on their color properties. In contrast, Blob Analysis is a technique used to detect and analyze the connected regions or blobs in an image. It is used to segment an image into regions of interest based on their shape, size, and other features.

The reason we use Blob Analysis is that it has several advantages over the Color Hough Transform in certain situations. Blob analysis can be faster than CHT for some types of image analysis tasks. While CHT requires a lot of computational resources to identify color regions in an image, blob analysis only needs to analyze connected regions. Blob analysis is also more robust to noise and image artifacts than CHT. CHT can be sensitive to variations in illumination, color, and texture, which can lead to false detections. Additionally, blob analysis can be applied to a wide range of shapes and sizes, whereas CHT is primarily used for detecting circular or elliptical shapes. Blob analysis often requires less parameter tuning than CHT. CHT requires careful tuning of parameters, such as the size and shape of the circular/elliptical kernel used for detection, whereas blob analysis often requires fewer parameters.

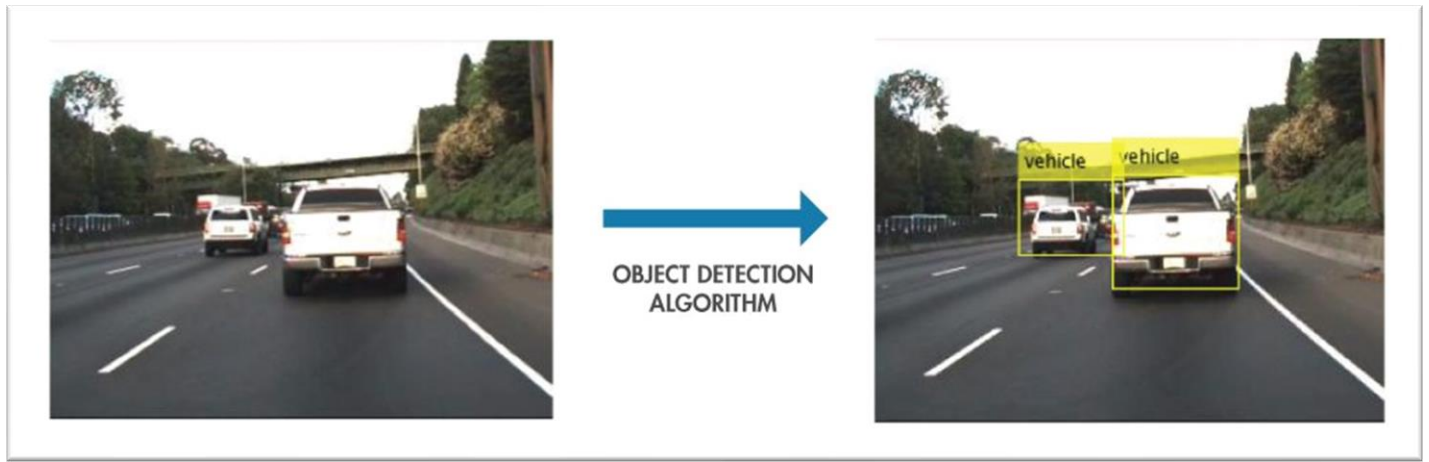


Fig. Example of Object Detection

DATA USED



The data we use for our task is an image containing yellow objects and a complex background, such as image of a mango tree.

METHODOLOGY

To achieve our objective, we have taken several steps and methods into account in order to obtain the desired result. These include:

1. Image Pre-processing
2. Color Processing
3. Gray Scaling, Filtering, and Enhancing
4. Blob analysis and Object Counting

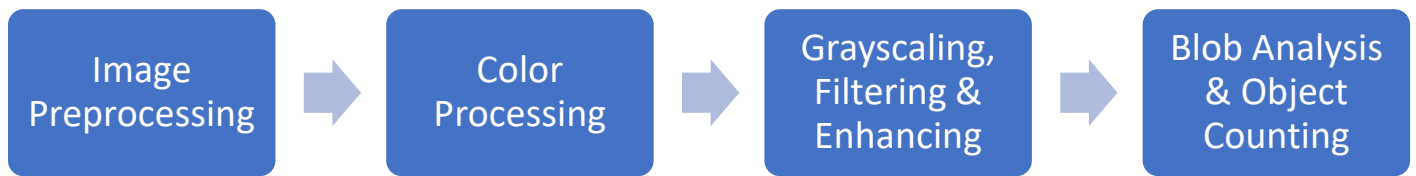


Fig. Block Diagram of Methodology

STEP 1: Image Preprocessing

First, the images are sent to the pre-processing block, where they undergo RGB adjustment to readjust the color of the object and background to make them lighter or darker as needed.

```
% Original Image  
Img = imread('Image.png');  
% RGB Adjustment  
RGBadjust = imadjust(A, [118/255 82/255 0; 254/255 247/255 204/255], []);
```



Original Image

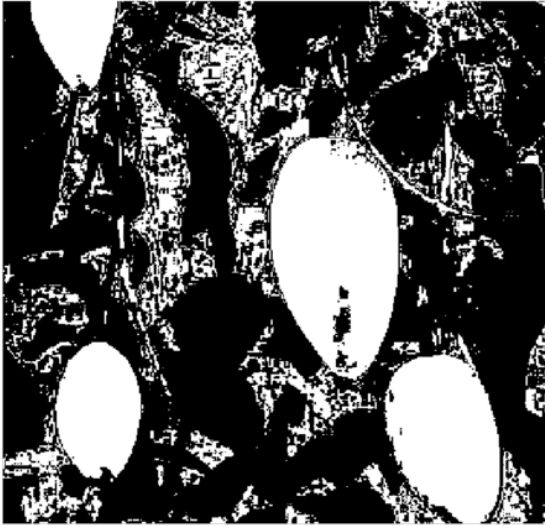


After RGB Adjustment

STEP 2: Color Processing

During color processing, unrelated colors are eliminated, leaving only the yellow color of the mango. This is achieved by comparing the current pixel's RGB value with the default RGB value of the mango. If the detected color is not related to the mango, the current pixel's RGB value is changed to 0, which appears as black. After color processing, the resulting image contains only the clear mango and the leaves, which have the same color as the mango.

```
I = rgb2hsv(RGBadjust);  
  
% Define thresholds for 'Hue'.  
channel1Min = 0.965;  
channel1Max = 0.188;  
  
% Define thresholds for 'Saturation'  
channel2Min = 0.000;  
channel2Max = 1.000;  
  
% Define thresholds for 'Value'  
channel3Min = 0.000;  
channel3Max = 1.000;  
  
% Create mask based on chosen histogram thresholds  
BW = ( (I(:,:,1) >= channel1Min) | (I(:,:,1) <= channel1Max) ) & ...  
      (I(:,:,2) >= channel2Min ) & (I(:,:,2) <= channel2Max) & ...  
      (I(:,:,3) >= channel3Min ) & (I(:,:,3) <= channel3Max);  
  
% Initialize output masked image based on input image.  
maskedRGBImage = RGBadjust;  
  
% Set background pixels where BW is false to zero.  
maskedRGBImage(repmat(~BW,[1 1 3])) = 0;
```



Mask



Masked RGB Image

STEP 3: Gray-Scaling, Filtering & Enhancing

Next, the image is converted to grayscale and a median filter is applied to remove small pixels and smooth the image. The median filter is effective at eliminating small pixels and enhancing the clarity of object edges. The resulting image is then converted to binary, and another object elimination step is applied to remove objects that consist of less than 200 pixels in a group of objects.

```
% Grayscale the Image
Gray = rgb2gray(maskedRGBImage);

% Medium Filter
filteredImg = medfilt2(Gray);
threshold = graythresh(filteredImg);
binaryImg = im2bw(filteredImg, threshold);

% Eliminating Objects under 200px
binaryImg = bwareaopen(binaryImg, 200);

% Perform morphological operations to enhance the binary image
se = strel('disk', 10);
binaryImg = imclose(binaryImg, se);
binaryImg = imfill(binaryImg, 'holes');
binaryImg = imopen(binaryImg, se);
```



Grayscaled Image



Binary Image



Eliminating Binary Objects under
200px



Enhancing Binary Image

STEP 4: Blob Analysis & Counting Objects

Blob Analysis is a fundamental technique of machine vision based on the analysis of consistent image regions. As such, it is a tool of choice for applications in which the objects being inspected are clearly discernible from the background. With a diverse set of Blob Analysis methods, we can come up with tailored solutions for a wide range of visual inspection problems.

In blob analysis, a blob is defined as a group of connected pixels with similar properties such as color, intensity, or texture. The properties of these blobs can be analyzed to extract information such as size, shape, orientation, and location of the objects in the image. Blob analysis can be used in a variety of applications such as object counting, object tracking,

defect detection, and quality control in manufacturing. It is particularly useful in applications where the objects of interest have a consistent shape or size and can be clearly distinguished from the background.

```
% Perform blob analysis to get the properties of the mangoes
blobAnalysis = regionprops(binaryImg, 'Area', 'BoundingBox', 'Centroid');
numMangoes = length(blobAnalysis);

% Draw bounding boxes around the mangoes
figure;
imshow(Img);
hold on;
for i = 1:numMangoes
    rectangle('Position', blobAnalysis(i).BoundingBox, 'EdgeColor', 'r',
        'LineWidth', 2);
end
title(sprintf('%d mango(es) detected', numMangoes));
```

FINAL RESULT



CONCLUSION

Based on the methods and techniques employed in this study, it is possible to accurately count the number of yellow objects, specifically mangoes, in an image with a complex background. By utilizing image pre-processing, color processing, gray scaling, filtering and enhancing, and blob analysis, we were able to isolate the mangoes from the background and accurately count them. This methodology can have practical applications in agriculture and food industry for crop yield estimation and quality control. Overall, the results of this study demonstrate the effectiveness of image analysis techniques in solving real-world problems and improving efficiency in various industries.

REFERENCES

- [1] R. Hussin, M. Rizon Juhari, Ng Wei Kang, R.C. Ismail, A. Kamarudin, Digital Image Processing Techniques for Object Detection from Complex Background Image, Procedia Engineering, Volume 41, 2012.
- [2] Dembele, F. Object Detection using Circular Hough Transform

————— *** —————